



Tema 3: Librerías/Componentes de Spark

Rafael Pastor Vargas
Andrés Duque Fernández
SCC-ETSI Informática-UNED

LIBRERÍAS/COMPONENTES DE SPARK

- Librerías disponibles en Spark.
- Gestión de datos al modo SQL: Spark SQL.
- **Machine Learning en Spark: Spark MLlib.**
- Procesamiento de gráfos: Spark GraphX.
- Introducción a Spark Streaming.

MACHINE LEARNING EN SPARK

- **Large Scale Machine Learning**
- Posibles usos
  - Monitorización de la seguridad y detección del fraude
  - Optimizaciones operacionales
  - Recomendación de productos (Marketing)
  - Segmentación de clientes
- Librería con los algoritmos soportados: **MLlib** (*deprecated 3.0*)
  - Inicialmente concebida para dar soporte a algoritmos ML, sin tener en cuenta flujos de trabajo (Data Pipelines)
  - Basado en RDD
  - Implementaciones
    - **org.apache.spark.ml** / org.apache.spark.mllib (Scala)
    - **pyspark.ml** / pyspark.mllib (Python)

spark.ml package

- Overview: estimators, transformers and pipelines
- Extracting, transforming and selecting features
- Classification and Regression
- Clustering
- Collaborative filtering
- Advanced topics

spark.mllib package

- Data types
- Basic statistics
- Classification and regression
- Collaborative filtering
- Clustering
- Dimensionality reduction
- Feature extraction and transformation
- Frequent pattern mining
- Evaluation metrics
- PMML model export
- Optimization (developer)

MLLIB: FUNCIONES ESTADÍSTICAS

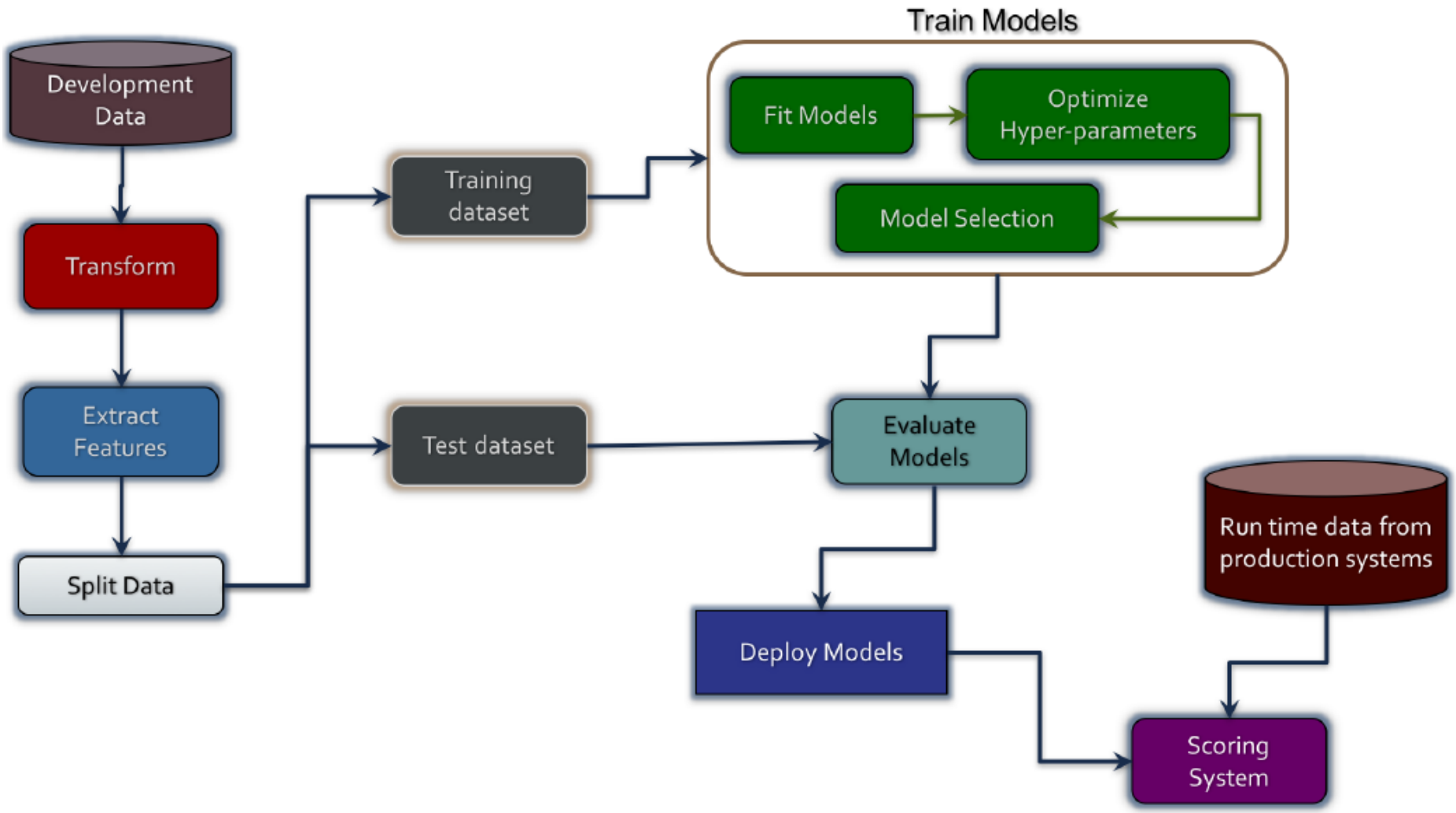
Algorithm	Feature	Notes
Basic statistics	Summary statistics	Here mean, stdev, count, max, min, and numNonZeros are all part of `dataframe.count()`, `dataframe.describe()`, and sql.functions
	Correlations and covariance	Here, sql.functions are invoked as `dataframe.stat.corr`(0 and cov)
	Stratified sampling	This provides two methods, `sampleBykey` and `sampleByKeyExact`, with and without replacement
	Hypothesis testing	This is to test the statistical significance, Pearson's *chi-squared* test for goodness of fit and independence tests
	Tests of data streams for A/B testing	Since 1.6.0, these are used to check the capability of both Welch's and Student's respective two-sample t-test over streams
	Random data generation	Generate data using functions such as Normal, Poisson, Exponential, gamma, and lognormal

MLLIB: ALGORITMOS IMPLEMENTADOS

Regression	Linear models	This includes linear regression: Least Square, Lasso, and the Ridge regression
Classification	Binary classification	This includes logistic regression, SVM, decision trees, random forests, gradient boosted trees, and naïve Bayes
	Multiclass classification	This includes logistic, decision trees, naïve Bayes, and random forests
Recommendation	Collaborative filtering	This includes alternating between least squares

Clustering	K-means	This includes K-means, Gaussian Mixture, and Streaming K-means
Dimensionality Reduction	SVD and PCA	
Feature Extraction and Transformation	TF-IDF, Word2Vec StandardScaler, andNormalizer	
Optimization	Gradient Descent, SGD, and L-BFGS	
Frequent pattern mining	The FP-growth Algorithm	
Model import/export		

ML PIPELINES

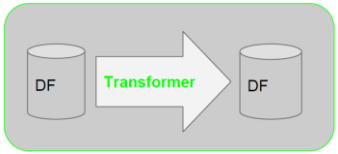


IMPLEMENTACIÓN DEL WORFLOW: CLASES

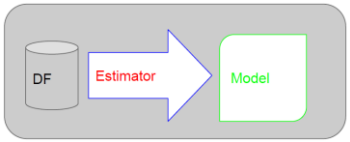
Workflow Stage	ML Feature	Notes
Transfom	org.apache.spark.ml.feature	Uses SQL functions such **assplit**
Extract Features	org.apache.spark.ml.feature	Uses Binarizer, Bucketizer, Normalizer, PAC, oneHotEncoder, Tokenizer, StandardScaler, StopWordsRemover, and Word2Vec
Split Data	org.apache.spark.ml.tuning org.apache.spark.sql.functions.split	Uses TrainValidationSplit
Fit Model	org.apache.spark.ml.regression, org.apache.spark.ml.recommendation	The machine learning algorithms live here
Model Optimization	org.apache.spark.ml.tuning	This is done through CrossValidator, and ParamGridBuilder
Model Evaluation	org.apache.spark.ml.evaluation.RegressionEvaluator	This calculates RMSE and MSE
Model Deployment	org.apache.spark.ml.util	Uses MLReader and MLWriter

ML PIPELINES (1/2)

- Abstracciones
  - Transformer
    - Su algoritmo usa los datos de un Dataframe/Dataset para construir otro
    - Implementa el método **transform()**

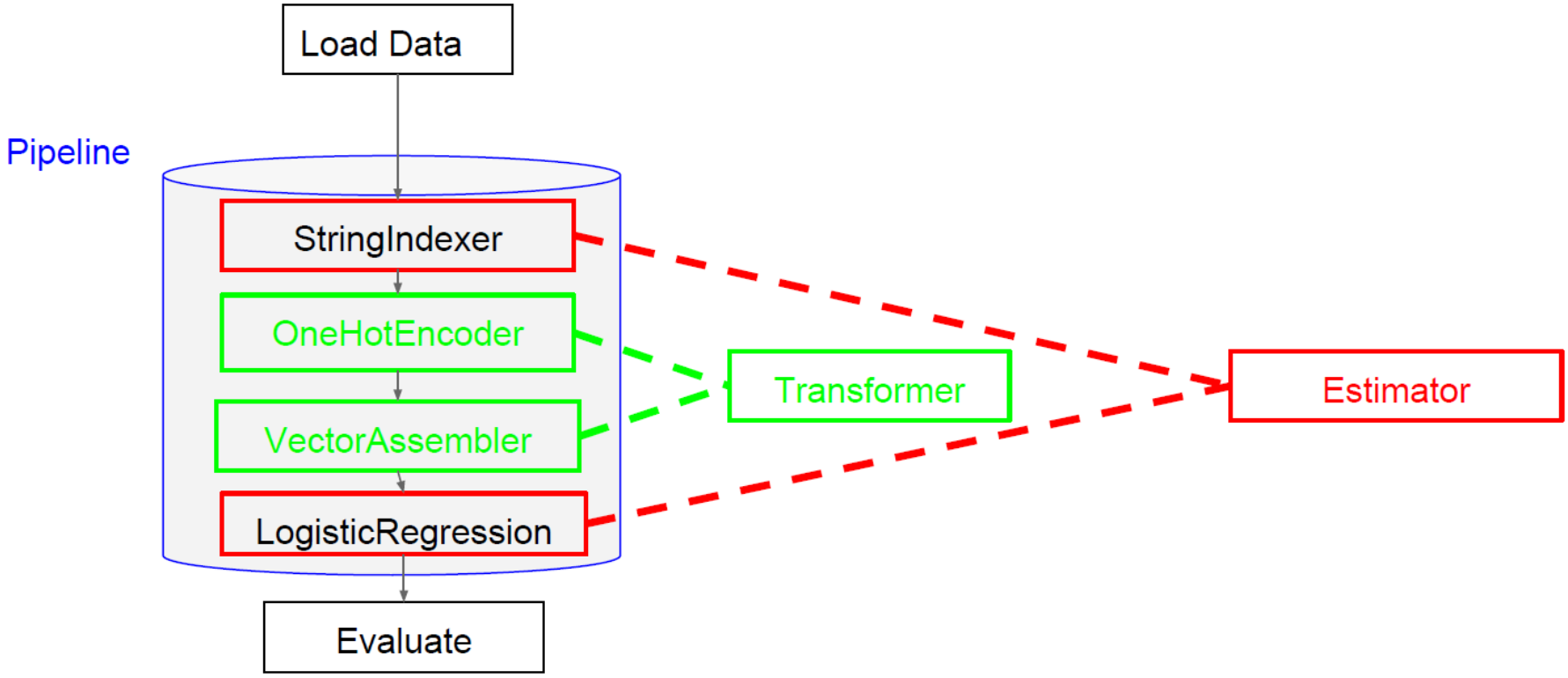


  - Estimator
    - Su algoritmo usa los datos del Dataframe/Dataset para devolver un modelo
    - Implementa el método **fit()**



  - Pipeline
    - Es una cadena de ejecución de transformadores/estimadores
    - Se considera un estimador
  - Evaluator
    - Calcula la precisión de estimación del modelo (métricas)
    - Implementa el método **evaluate()**
  - Params

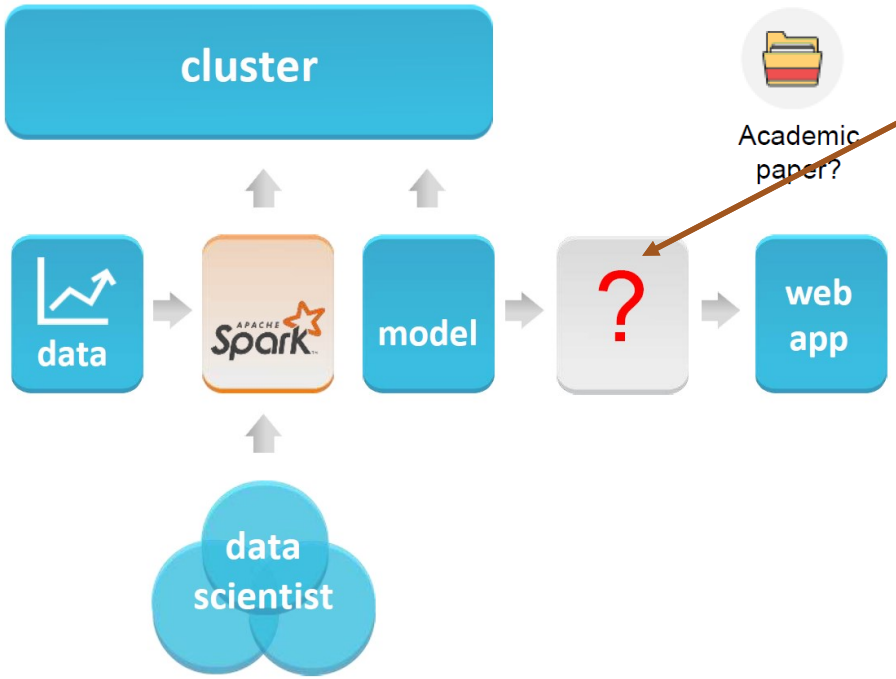
ML PIPELINES (2/2)

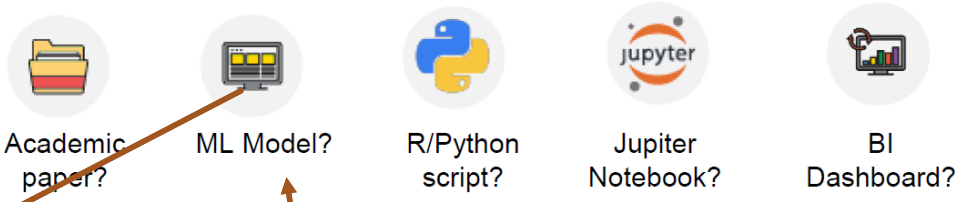


Y AHORA QUE?

- Gartner has identified predictive analytics as the most important trend in B2B trade intelligence, and predicts that by 2020:
  - "Predictive and prescriptive analysis will attract 40% of companies' net investment in Business Intelligence and analytical tools."

- El producto generado es un modelo !!!






```scala
val tokenizer = new Tokenizer()
  .setInputCol("text")
  .setOutputCol("words")

val hashingTF = new HashingTF()
  .setNumFeatures(1000)
  .setInputCol(tokenizer.getOutputCol)
  .setOutputCol("features")

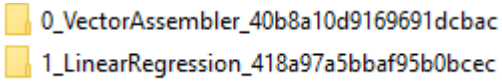
val lr = new LogisticRegression()
  .setMaxIter(10)
  .setRegParam(0.001)

val pipeline = new Pipeline()
  .setStages(Array(tokenizer, hashingTF, lr))

val model = pipeline.fit(training)
model.write.save("/tmp/spark-model")
```

SERIALIZACIÓN DE MODELOS (1/2)

- Almacenar el modelo para poder usarlo después
  - Python ➔ model.write().save("models/lr")
- Stages



- Metadata

```
class : org.apache.spark.ml.PipelineModel
timestamp : 1517339419169
sparkVersion : 2.2.1
uid   : PipelineModel_46259525613ec40d28c4
▼ paramMap {1}
  ▼ stageUids [2]
      0  : VectorAssembler_40b8a10d9169691dcbac
      1  : LinearRegression_418a97a5bbaf95b0bcec
```

```
class : org.apache.spark.ml.regression.LinearRegressionModel
timestamp : 1517339419460
sparkVersion : 2.2.1
uid   : LinearRegression_418a97a5bbaf95b0bcec
▼ paramMap {11}
    maxIter : 10
    fitIntercept : ☑ true
    aggregationDepth : 2
    standardization : ☑ true
    labelCol : mpg
    elasticNetParam : 0.8
    regParam : 0.3
    solver : auto
    predictionCol : prediction
    tol   : 0.000001
    featuresCol : features
```

SERIALIZACIÓN DE MODELOS (2/2)

- Los datos de las etapas se guardan en formato Parquet

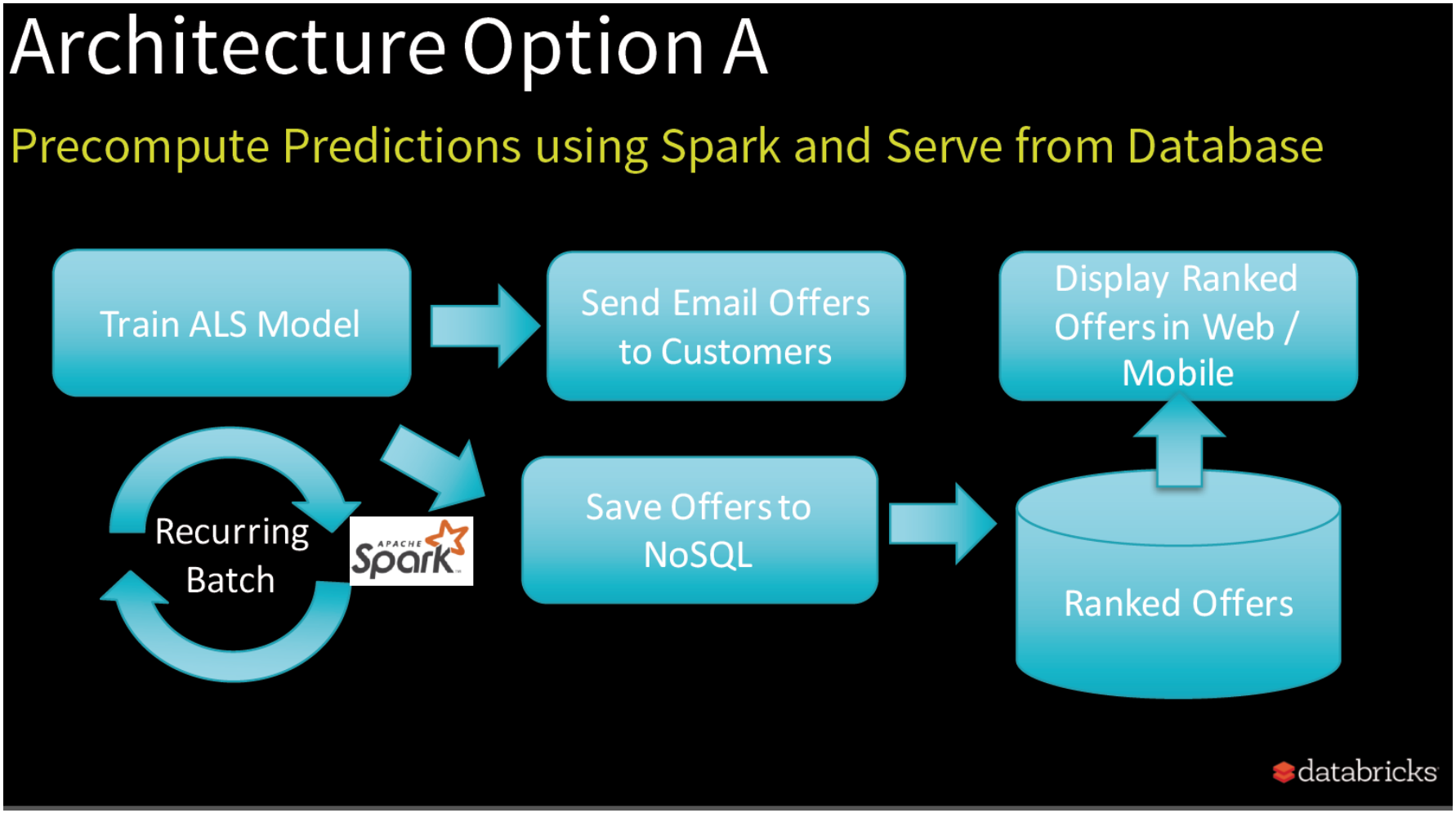
```
spark.read.parquet("models/lr/stages/1_LinearRegression_418a97a5bbaf95b0bcec/data/").show()
```

```
+-----------------+--------------------+
|        intercept|        coefficients|
+-----------------+--------------------+
|29.15726405117587|[-0.0109727485253...|
+-----------------+--------------------+
```

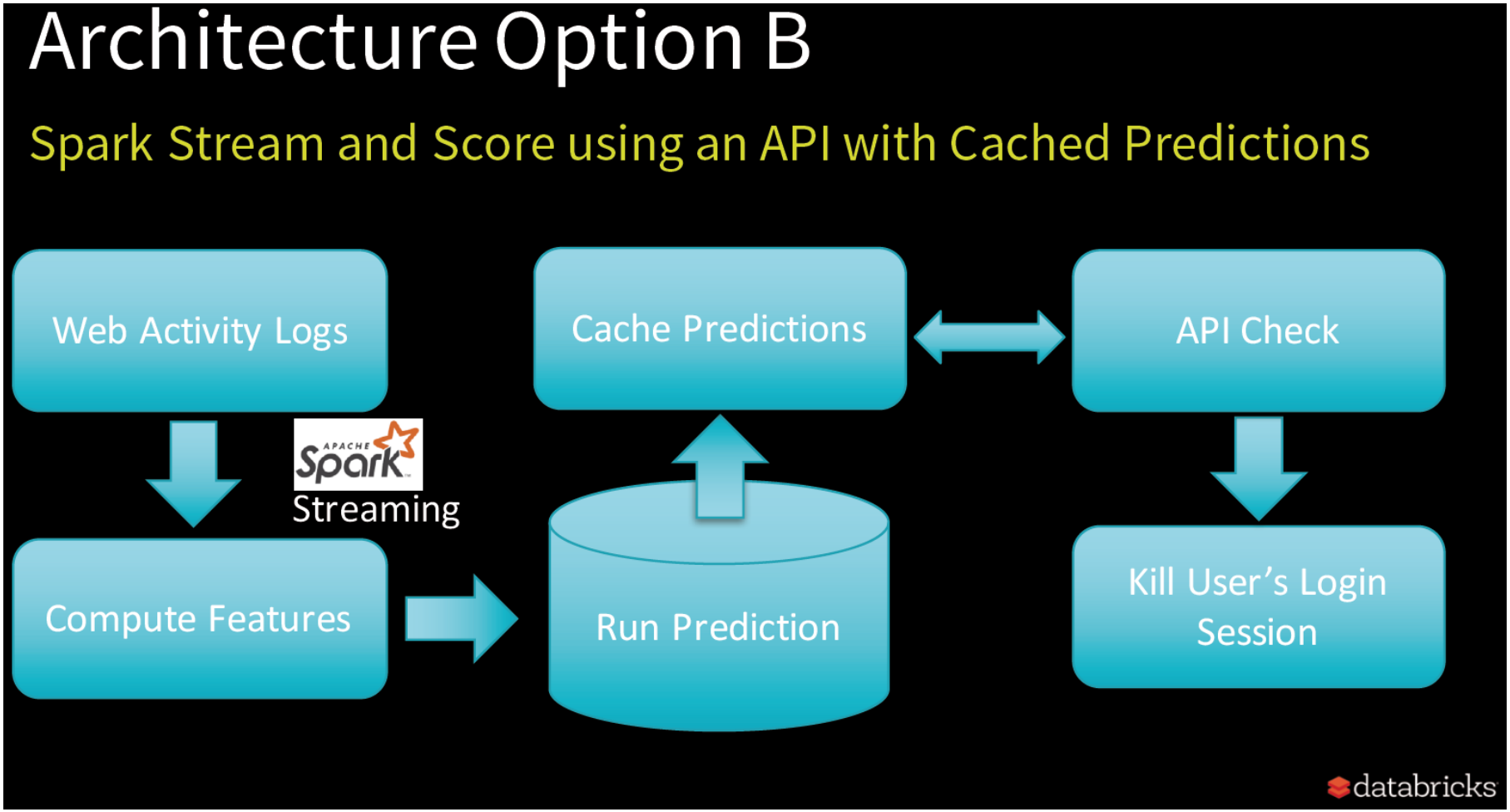
- Se pueden recuperar fácilmente

```
# Load model
from pyspark.ml import PipelineModel
modelLoaded = PipelineModel.load("models/lr")
```

ARQUITECTURAS DE DESPLIEGUE (1/3)



ARQUITECTURAS DE DESPLIEGUE (2/3)



ARQUITECTURAS DE DESPLIEGUE (3/3)

